

Київський національний університет ім. Тараса Шевченка

Кафедра мережевих та інтернет технологій

Лабораторна робота № 5

Дисципліна: Системне програмування

Тема: Динамічна ідентифікація типів (Рефлексія).

Виконав: Студент групи МІТ-41

Полюхович Андрій

Варіант: 4

Завдання:

1. Створити новий проєкт за шаблоном Windows Forms Application.
2. На форму додати елемент керування TreeView.
3. Описати клас, забезпечивши наявність:
 - 4-х властивостей різних типів, одна з яких мала б тип колекції;
 - 3-х методів;
 - 2-х конструкторів.
4. Описати метод, який би виводив перелік всіх властивостей (разом з типом та значенням) об'єкта описаного класу у TreeView.
 - class Мобільний.

Виконання:

У ході виконання лабораторної роботи було створено проєкт Windows Forms. Логіку програми розділено на два файли згідно з принципами об'єктно-орієнтованого програмування: опис класу винесено в окремий файл MobilePhone.cs, а логіку взаємодії з інтерфейсом та рефлексією залишено у Form1.cs.

1. Опис досліджуваного класу (Файл MobilePhone.cs) Було створено клас MobilePhone, який містить 4 властивості різних типів (зокрема колекцію List<string>), 2 конструктори (без параметрів та з параметрами) і 3 методи, як того вимагає завдання.

файл **MobilePhone.cs**:

```
using System.Collections.Generic;
```

```
namespace lab
```

```
{
```

```
    public class MobilePhone
```

```
    {
```

```
        public string Model { get; set; }
```

```
        public double Price { get; set; }
```

```
        public int Year { get; set; }
```

```

        public List<string> Features { get; set; }

        public MobilePhone()
        {
            Features = new List<string>();
        }

        public MobilePhone(string model, double price, int year,
List<string> features)
        {
            Model = model;
            Price = price;
            Year = year;
            Features = features;
        }

        public void MakeCall(string phoneNumber) { }
        public void SendSMS(string textMessage) { }
        public void ChargeBattery() { }
    }
}

```

2. Реалізація рефлексії та інтерфейсу (Файл Form1.cs) На форму додано елементи Button та TreeView. Метод AnalyzeWithReflection динамічно отримує тип переданого об'єкта та витягує всю інформацію про нього за допомогою Reflection API.

файл **Form1.cs**:

```

using System;
using System.Collections.Generic;
using System.Reflection;
using System.Windows.Forms;

namespace lab
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            private void buttonLoad_Click(object sender, EventArgs e)
            {
                MobilePhone myPhone = new MobilePhone("Samsung Galaxy",
35000.50, 2024,
                new List<string> { "NFC", "5G", "OLED Екран" });

                AnalyzeWithReflection(myPhone);
            }

```

```

        private void treeView1_AfterSelect(object sender,
TreeViewEventArgs e)
        {
        }

        private void AnalyzeWithReflection(object obj)
        {
            treeView1.Nodes.Clear();

            Type t = obj.GetType();

            TreeNode root = new TreeNode($"Клас: {t.Name}");
            treeView1.Nodes.Add(root);

            TreeNode propsNode = new TreeNode("Властивості
(Properties)");
            foreach (PropertyInfo prop in t.GetProperties())
            {
                object val = prop.GetValue(obj);
                string valStr = val is List<string> list ?
string.Join(", ", list) : val?.ToString();

                propsNode.Nodes.Add($"{prop.Name}
({prop.PropertyType.Name}) = {valStr}");
            }
            root.Nodes.Add(propsNode);

            TreeNode methodsNode = new TreeNode("Методи (Methods)");
            foreach (MethodInfo method in
t.GetMethods(BindingFlags.Public | BindingFlags.Instance |
BindingFlags.DeclaredOnly))
            {
                ParameterInfo[] parameters = method.GetParameters();
                string paramStr = string.Join(", ",
Array.ConvertAll(parameters, p => $"{p.ParameterType.Name}
{p.Name}"));

                methodsNode.Nodes.Add($"{method.ReturnType.Name}
{method.Name}({paramStr})");
            }
            root.Nodes.Add(methodsNode);

            TreeNode constrNode = new TreeNode("Конструктори
(Constructors)");
            foreach (ConstructorInfo constructor in
t.GetConstructors())
            {
                ParameterInfo[] parameters =
constructor.GetParameters();
                string paramStr = string.Join(", ",
Array.ConvertAll(parameters, p => $"{p.ParameterType.Name}
{p.Name}"));

                constrNode.Nodes.Add($"{t.Name}({paramStr})");
            }
        }
    }
}

```

```

    }
    root.Nodes.Add(constrNode);

    treeView1.ExpandAll();
}
}
}

```

Результат роботи програми :

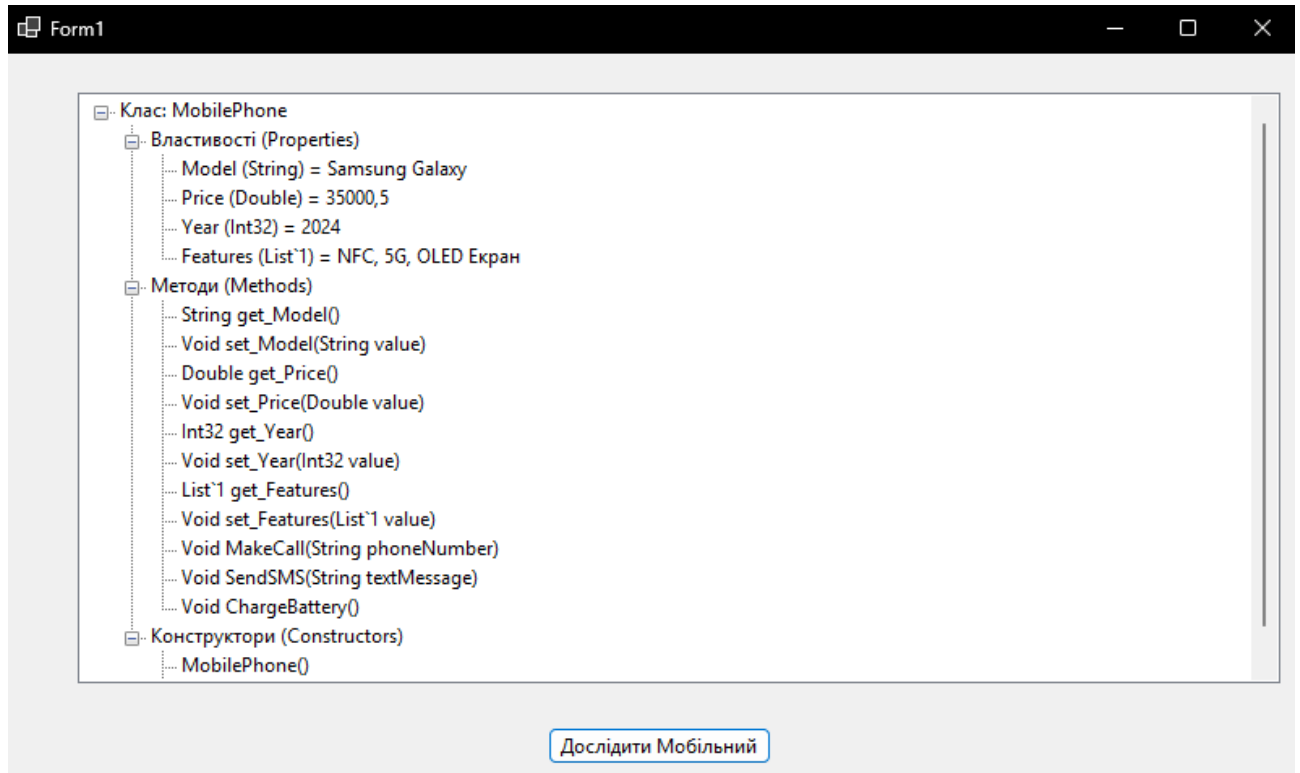


Рисунок 5.1 – Результат роботи

Аналіз роботи програми :

Отримання типу: Програма успішно використовує метод `GetType()`, отримуючи об'єкт класу `Type` (ядро підсистеми рефлексії), що дозволяє досліджувати об'єкт під час виконання.

Аналіз властивостей: За допомогою методу `GetProperties()` рефлексія динамічно витягує масив усіх публічних властивостей об'єкта (`Model`, `Price`, `Year`, `Features`). Метод `GetValue()` дозволяє прочитати їхні фактичні значення безпосередньо з об'єкта.

Аналіз методів: Виклик `GetMethods()` дозволив сформувати список методів класу. Завдяки масиву `ParameterInfo[]` програма також динамічно визначає і відображає типи та імена параметрів, які приймають ці методи (наприклад, `String phoneNumber` для методу `MakeCall`). Примітка: у списку методів також відображаються системні аксесори `get_` та `set_`, оскільки на рівні IL-коду властивості реалізуються саме через ці методи.

Аналіз конструкторів: Метод `GetConstructors()` успішно ідентифікував два описані в класі конструктори та їхні параметри

Висновок

У результаті виконання лабораторної роботи було успішно засвоєно механізм рефлексії в C#. Практично доведено, що простір імен `System.Reflection` є надзвичайно потужним інструментом, який дозволяє динамічно (під час виконання програми) виявляти структуру типів даних, отримувати списки їхніх полів, властивостей, методів та конструкторів. Розроблена програма повністю відповідає вимогам завдання, коректно ідентифікує всі складові класу `MobilePhone` та візуалізує їх у компоненті `TreeView`.